

Image Compression Project Report

Xingyu Liu 400301133

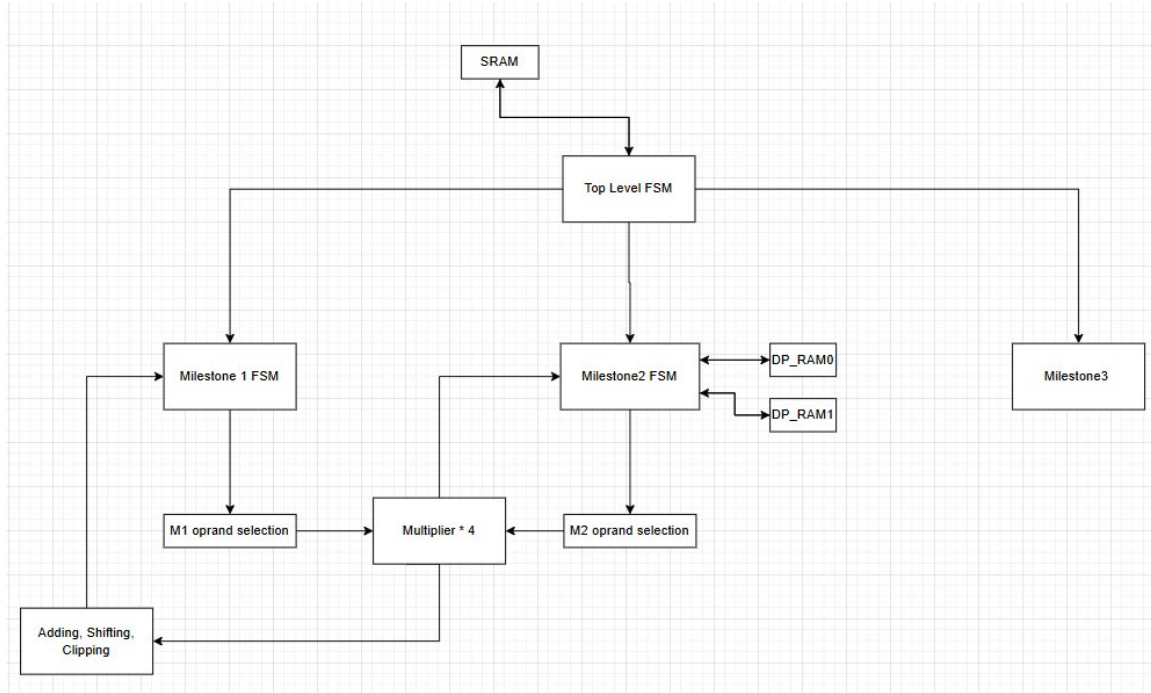
Introduction

This project is related to Image compression by using DE2-115 and SystemVerilog. The maximum supported image size is 640*480. The original image .ppm file will be delivered to the Altera DE2-115 board via UART from PC. The original (RGB) image data will be stored in the external SRAM. The goal is to design a hardware digital circuit that can:

- read the original RGB data from external SRAM, do the color space conversion process and downsampling to obtain the YUV(or YCbCr) data and write back to the external SRAM (Milestone 1),
- and then read YUV(or YCbCr) data from external SRAM, do the signal transform (Discrete Cosine Transform) process and write the post-DCT YUV data back to the external SRAM (Milestone 2),
- then read the post-DCT YUV data from external SRAM and do the quantization and lossless coding process (Milestone 3).

The digital circuit is designed with SystemVerilog in Quatus, and verified the circuit with testbench in Modelsim.

Design Structure



Implementation Details

Milestone 1

For milestone1, in general, FSM was used to read RGB value from external SRAM, do the calculation to get Y, U', V' (color space conversion), and use 7-taps to get U and V (downsampling), and write YUV value back to external SRAM.

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.257 & 0.504 & 0.098 \\ -0.148 & -0.291 & 0.439 \\ 0.439 & -0.368 & -0.071 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 16 \\ 128 \\ 128 \end{bmatrix}$$

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = (int) \frac{1}{65536} \left(\begin{bmatrix} 16843 & 33030 & 6423 \\ -9699 & -19071 & 28770 \\ 28770 & -24117 & -4653 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 32768 \\ 32768 \\ 32768 \end{bmatrix} \right) + \begin{bmatrix} 16 \\ 128 \\ 128 \end{bmatrix} \quad (1)$$

$$U[j] = 0.043U'[2j-5] - 0.102U'[2j-3] + 0.311U'[2j-1] + 0.5U'[2j] + 0.311U'[2j+1] - 0.102U'[2j+3] + 0.043U'[2j+5]$$

$$V[j] = (int) \frac{1}{312} (22U'[2j-5] - 52U'[2j-3] + 159U'[2j-1] + 256U'[2j] + 159U'[2j+1] - 52U'[2j+3] + 22U'[2j+5] + 256)$$

Fig. Formula used by calculating Y, U', V', and U, V.

Lead In Case:

Lead In Case					
state (33 in total)	0-4	5-11	12-18	19-25	26-32
	Read RGB 0,1	Read RGB 2,3	Read RGB 4,5	Read RGB 6,7	Read RGB 8,9
		CSC 0,1	CSC 2,3	CSC 4,5	CSC 6,7
				DS U0V0	DS U1V1
			Write Y0Y1	Write Y2Y3	Write Y4,Y5

In lead in case, shift registeres was initiated by fill the u'0 and v'0 to all the 4 shift register sets (u odd shft register set, v odd shift register set, u even shift register set, v even shift register set). All 4 shift register sets will shift once in every round of common case.

Common Case:

state (7 in one round)	0-6
	Read RGB pair (two values in one pair)
	CSC 2 pairs
	DS 1
	Write 2 Y values
	Write 2 U values
	(next round write 2 V values)
	jump out to lead out state when ==image width - 4
	(when the second last pair of RGB data finished readed from SRAM and the last pair of RGB data will be read in the next round, jump to lead out state. For example, when processing a image with size 320*240, if this round of cc read RGB 316,317, jump out)

Multipliers behavior (In one round of common case):

state	0	1	2	3	4	5	6
multiplier 0	Y even	U' even	V' even	U' odd	V' odd	DS U	IDLE
multiplier 1	Y even	U' even	Y odd	U' odd	V' odd	DS V	IDLE

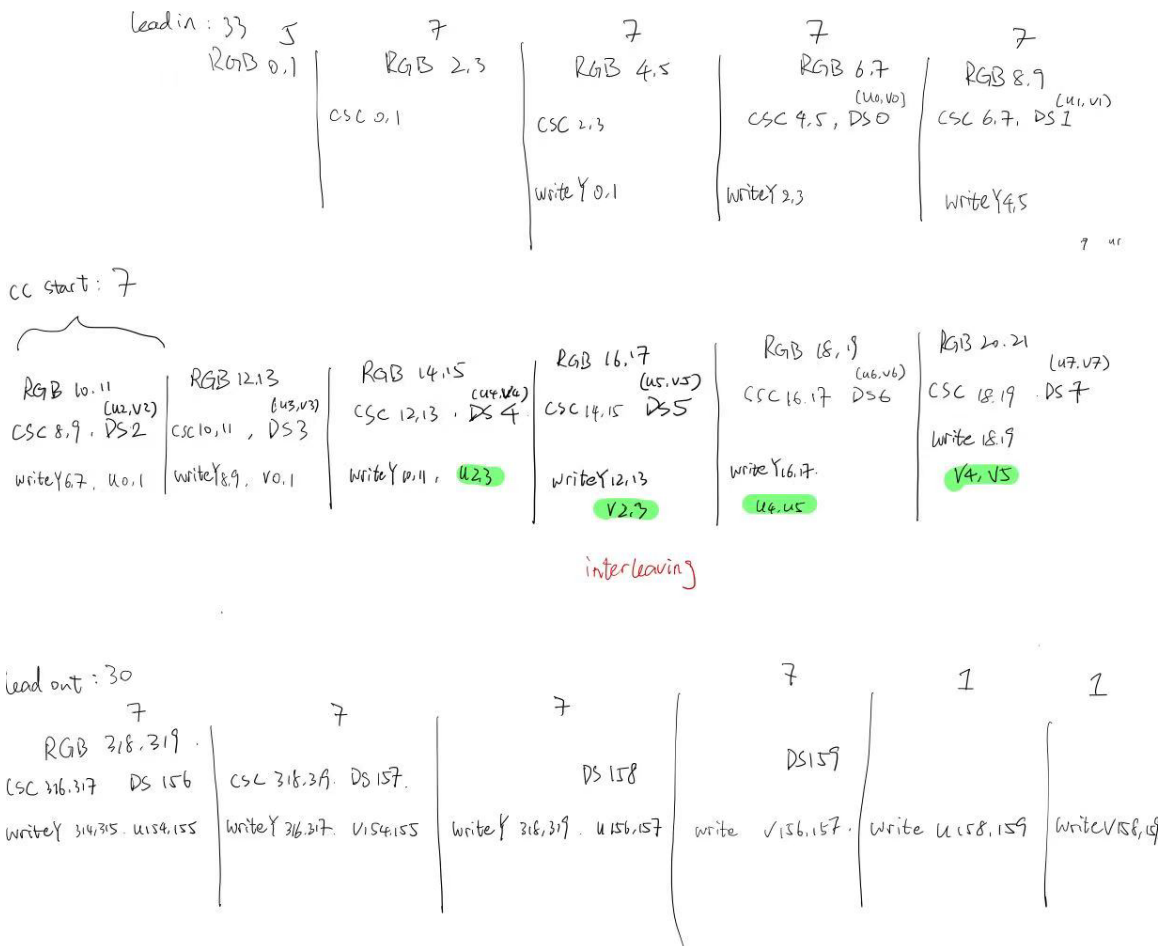
multiplier 2	Y even	V' even	Y odd	U' odd	DS U	DS V	IDLE
multiplier 3	U' even	V' even	Y odd	V' odd	DS U	DS V	IDLE

Lead Out Case:

Lead Out Case (30 in total) (use 320*240 image as example)	0-6	7-13	14-20	21-27	28	29
	RGB 318,319					
	CSC 316,317	CSC 318,319				
	DS 156	DS 157	DS 158	DS 159		
	write Y 314,315	write Y 316,317	write Y 318,319			
	write U154,155	write V154,155	write u156,157	write v156,157	write u158,159	write v158,159

A row counter is used for count how many rows has been processed. If the row counter is equal to the maximum number of rows, then jump to idle state of M1 FSM, and use “done1” flag to tell the top-level FSM that Milestone 1 has finished. Else it will jump back to the first lead in state to start to process a new row of pixel.

An example of the pipeline when processing 320*240 image:



Defined Registers:

Module	Register Name	Bits	Description
Milestone 1	done1	1	A flag to indicate milestone 1 or color space conversion and downsampling process is finished
Milestone 1	SRAM_address	20	Store the external SRAM location
Milestone 1	SRAM_write_data	16	Store the value that will write to SRAM
Milestone 1	SRAM_we_n	1	SRAM write enable (active low)
Milestone 1	SRAM_address_RGB	20	Store the SRAM location of RGB data
Milestone 1	SRAM_address_Y	20	Store the SRAM location of Y data
Milestone 1	SRAM_address_U	20	Store the SRAM location of U data
Milestone 1	SRAM_address_V	20	Store the SRAM location of V data
Milestone 1	SRAM_data_R[1:0]	8	2 registeres that store the R value of the RGB data pairs which read from SRAM
Milestone 1	SRAM_data_G[1:0]	8	2 registeres that store the R value of the RGB data pairs which read from SRAM
Milestone 1	SRAM_data_B[1:0]	8	2 registeres that store the R value of the RGB data pairs which read from SRAM
Milestone 1	Y_buf[1:0]	8	2 registeres that store the Y value after it has been calculated by multipliers
Milestone 1	Up_buf[1:0]	8	2 registeres that store the U' value after it has been calculated by multipliers
Milestone 1	Vp_buf[1:0]	8	2 registeres that store the V' value after it has been calculated by multipliers
Milestone 1	U_buf[1:0]	8	2 registeres that store the U value after it has been calculated by multipliers
Milestone 1	V_buf[2:0]	8	3 registeres that store the V value after it has been calculated by multipliers
Milestone 1	ACC_reg_0	32	Accumulate Register that store the result of the multiplier
Milestone 1	ACC_reg_1		

Milestone 1	U_odd_shift_Reg[5:0]	8	4 sets of shift registers. U/V odd shift registers are 6 registers each. U/V even shift registers are 3 registers each
Milestone 1	V_odd_shift_Reg[5:0]		
Milestone 1	U_even_shift_Reg[2:0]		
Milestone 1	V_even_shift_Reg[2:0]		
Milestone 1	select0	5	Multipliers Operands will choose the correct value based on the value of select. (select0 is for multiplier1's operand, select1 is for multiplier2's operand chosen, and so on)
Milestone 1	select1		
Milestone 1	select2		
Milestone 1	select3		
Milestone 1	Row_counter	10	A counter that count how many rows of the pixels are finished processing
Milestone 1	Col_counter	11	A counter that count how many columns of the pixels are finished processing
Milestone 1	uv_write_flag	1	A flag that indicate if the current round should write 2 U data or 2 V data to external SRAM
Milestone 1	uv_buf_flag	1	A flag that indicate if the V data should be buffered for one round (7) of common case

Milestone 2

Here is the formula used for milestone 2 DCT process.

$$S' = CSC^T, [C]_{i,j} = \begin{cases} \sqrt{\frac{1}{8}} \cos\left(\frac{i\pi}{8}(j+0.5)\right) & i=0; j=0..7 \\ \sqrt{\frac{2}{8}} \cos\left(\frac{i\pi}{8}(j+0.5)\right) & i=1..7; j=0..7 \end{cases} \quad (2)$$

$$S' = (int) \frac{1}{65536} \left(C \times (int) \frac{1}{256} (S' C^T + 2^7 \mathbf{1}) + 2^{15} \mathbf{1} \right), [C]_{i,j} = \begin{cases} (int) \left(\sqrt{\frac{1}{8}} \cos\left(\frac{i\pi}{8}(j+0.5)\right) \times 4096 \right) & i=0; j=0..7 \\ (int) \left(\sqrt{\frac{2}{8}} \cos\left(\frac{i\pi}{8}(j+0.5)\right) \times 4096 \right) & i=1..7; j=0..7 \end{cases}$$

Fig. Formula for DCT process

In general, the data was processed in blocks. Every time a 8*4 block will be read from external SRAM, process this block and write the result 8*8 block back to external SRAM. For each block of data, it will be processed in 4 steps: Fetch S, Compute T (intermediate value), Compute S', Write S'. However, requirement of the project is to complete the DCT process within 25ms when processing the 640*480 image. If the process is Fetch S for first block, Compute T for first block, Compute S' for first block, Write S' for first block, then Fetch S for second block, Compute T for second block... 43ms is needed! Thus, in order to meet the project requirements,

Block

Mega state A
mega state B

K	Fs	CT	Cs'	Ws'			
K+1		Fs	CT	Cs'	Ws'		
K+2			Fs	CT	Cs'	Ws'	
K+3				Fs	CT	Cs'	Ws'

Fetch S state:

Common Case:

state	0	1	2	3	4	5	6	7
SRAM_write_enab	1	1	1	1				
SRAM_address					a320(+317)	a321	a322	a323
SRAM_read_data	d(y4,y5)	d(y6,y7)					d(y640,y641)	d
SRAM_write_data								
[4:0]SC_s					4+1	5+1	6+1	7+1
CB								
RB								
fs_buf		s04,s05(y4,y5)						s10,s11,(y640,y641)
DPRAM0								
DPO_Address_a (ac	0		1					
DPO_write_data_a	{Read_Reg, SRAM_read_data}(s0,0123)		{Read_Reg, SRAM_read_d ata}(s0,4567)					
DP0_Write_en_a (v	1	0	1	0				0

In DCT formula $S' = C * S * C^t$, $S * C^t$ is calculated in this state. In one round of common case, one row of S will be obtained from dual port RAM. Calculation process use butterfly optimization, because the value in DCT matrix is horizontally symmetrical (in row 1, the value on first column is equal to the value on last column. In row 2, the absolute value of the value on first column is equal to the absolute value of the value on last column, but one is positive and another is negative. The rest of the rows have the same pattern). Thus, instead of multiplied 8 times to get one T

data, we can add/subtract the value on the first column and last column in the same row first, then multiplied with the same DCT factor:

$$\begin{aligned} (S_{00} + S_{07}) \times C_{00}^T + \\ (S_{01} + S_{06}) \times C_{10}^T + \\ (S_{02} + S_{05}) \times C_{20}^T + \\ (S_{03} + S_{04}) \times C_{30}^T \end{aligned}$$

Thus, only 4 multiplications can get one T data. There are 4 multiplications in every state, so every state can contain one T data, and 8 states (one round of common case) will get one row of T data.

$$\begin{pmatrix} 1 & x & x & x & x \\ 1 & \dots & x & x & x \end{pmatrix}^T = \begin{pmatrix} 1 & \rightarrow \\ 1 & \rightarrow \\ 1 & \rightarrow \\ 1 & \rightarrow \end{pmatrix} \begin{pmatrix} 1 & \rightarrow \\ 2 & \rightarrow \\ 7 & \rightarrow \\ 2 & \rightarrow \end{pmatrix}$$

$$\begin{pmatrix} 1 & \dots & x & x & x \end{pmatrix} = \begin{pmatrix} 1 & \rightarrow \\ 1 & \rightarrow \\ 1 & \rightarrow \\ 1 & \rightarrow \end{pmatrix} \begin{pmatrix} 5 & \rightarrow \\ 6 & \rightarrow \\ 7 & \rightarrow \\ 8 & \rightarrow \end{pmatrix}$$

In order to save more cycles, in the first round, the first row of S will be read and store in the first set of registers (s0_reg_0 to s7_reg_0). In the second round, multipliers will start to do multiplication using the first row of S, while obtained the second

row of S and stored in the second set of registers (s0_reg_1 to s7_reg1), which will be used as multipliers input in the next round. Thus no extra cycles are needed for read and store the s value only. Since DCT matrix is passed by a MUX, so no register or cycle is needed for obtaining DCT factors.

After obtained the result of each T in 32 bits, write it directly back to the dual port RAM 1. The T data is stored from line 0 to line 64 in dual port RAM 1.

Compute S':

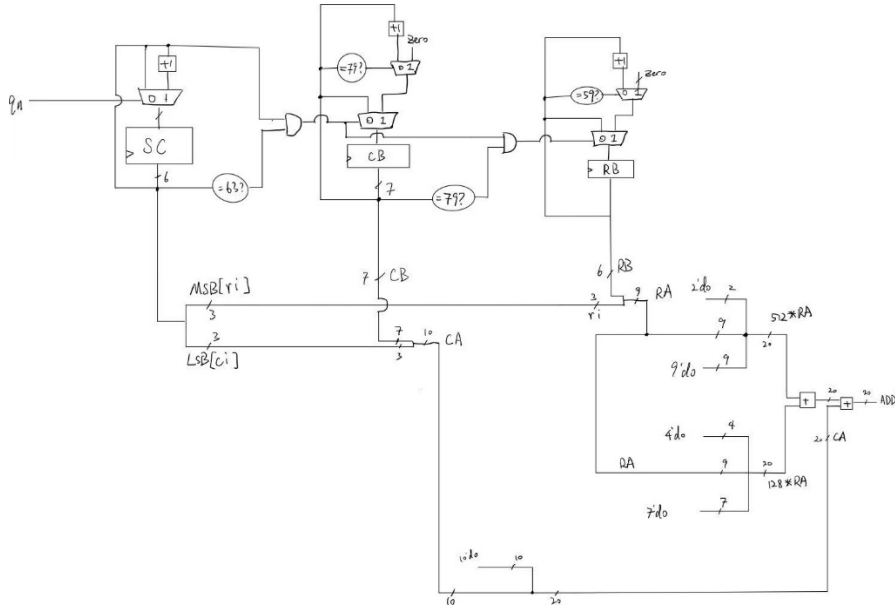
The Common case of Compute S' is similar to compute t, each state will get one s' data, and 8 state will get one row of s' data. 2 sets of registeres (t0_reg_0 to t7_reg_0 and t0_reg_1 to t7_reg1) are used in order to save more data reading cycles. The difference between compute T and Compute S' is in compute s', the formula is $S' = C * T$, which means every time a column of T will be obtained. However, T is stored in row (t00,t01,t02...t07,t10,t11...t77) in dual port RAM. Thus, in order to get correct T value, a 6 bits sample counter is used. Every time the sample will increase by 1, and the address of dual port RAM will be {1'b0,sc_t[2:0],sc_t[5:3]} which swapping the 3 MSB and 3 LSB of the sample counter, which make sure that every time sample counter increase by 1 means the row index is increased by 1. The S' data (in 32 bits) will be saved from line 16 to line 79 in dual port RAM 0.

Write S':

In this state, one 32 bits s' value is shifted by 16 bits and write to external SRAM in every clock cycle.

External SRAM Address Management in Fetch S and Write S' states:

Since the data was read from and write back to the external SRAM block by block, a address management logic is needed. Here is a general structure of circuit:



The address management for fetch s and write s' is similar. The difference is that a block of s is 8*4, whereas a block of s' is 8*8. Thus, the sample counter for s is 5 bit (3 MSB is row index, 2 LSB is column index). The sample counter for s' is 6 bits (3 MSB for ri and 2 LSB for ci). When read/write Y data, the maximum CB is 79. When read/write U/V data, the maximum CB is 39. When processing 640*480 image, the formula for address is:

$$Y = (RB \times 8 + ri) \times 320 + (CB \times 4 + ci) \quad U/V: (RB \times 8 + ri) \times 160 + (CB \times 4 + ci)$$

In SystemVerilog:

$$Y: \underbrace{\{3'd0, \{RB, SC[4:2]\} \ll 8\}}_{RA \times 256} + \underbrace{\{5'd0, \{RB, SC[4:2]\} \ll 6\}}_{RA \times 64} + \underbrace{\{11'd0, CB, SC[1:0]\}}_{CA}$$

$$U/V: \underbrace{\{4'd0, \{RB, SC[4:2]\} \ll 7\}}_{RA \times 128} + \underbrace{\{6'd0, \{RB, SC[4:2]\} \ll 5\}}_{RA \times 32} + \underbrace{\{11'd0, CB, SC[1:0]\}}_{CA}$$

In order to save multipliers, 320 is 256 + 64. *256 and *64 can be achieved by right shift 8 bits and 6 bits. However, this structure can only deal with 640*480 image, not other size of images. Thus, in order to fit all the size of images, the address is defined in the code: `SRAM_address <= sram_fs_addr_offset + ({11'd0, rb_s, sc_s[4:2]} * Y_FS_STRIDE) + {11'd0, cb_s, sc_s[1:0]}`;

Y_FS_STRIDE is a parameter. Thus, when this code is synthesized in quatus, the system will know that numbers like 640, 320, 160 stored in a parameter are magic numbers, which can be break into two parts and do right shifting instead of wasting multipliers to achieve multiplication.

Here is a proof that multiplying directly with a parameter will not waste multipliers (DSP block):

✓ I/O pins	131 / 529 (25 %)
-- Clock pins	1 / 7 (14 %)
-- Dedicated input pins	0 / 9 (0 %)
M9Ks	4 / 432 (< 1 %)
Total block memory bits	8,192 / 3,981,312 (< 1 %)
Total block memory implementation bits	36,864 / 3,981,312 (< 1 %)
Embedded Multiplier 9-bit elements	16 / 532 (3 %)
PLLs	1 / 4 (25 %)

The Compilation Report – Resource Usage Summary shows that 16 embedded multiplier 9-bit element was used. This does not violate the requirement that use only 4 32-bit multipliers, because 1 32-bit multiplier was formed by 4 9-bit DSP blocks, and 4 32-bit multipliers need 16 9-bit DSP blocks.

Defined Registers:

Module	Register Name	Bits	Description
Milestone 2	done2	1	A flag to indicate milestone 2 or DCT process is finished
Milestone 2	SRAM_address	20	Store the external SRAM location
Milestone 2	SRAM_write_data	16	Store the value that will write to SRAM
Milestone 2	SRAM_we_n	1	SRAM write enable (active low)
Milestone 2	sc_s	5	2 msb are row index, 3 lsb are column index for s (pre-dct) value in sram
Milestone 2	sc_sp_dp0	6	3 msb are row index, 3 lsb are column index for s' (post-dct) value in dpram
Milestone 2	sc_sp_sram	6	3 msb are row index, 3 lsb are column index for s' (post-dct) value in sram
Milestone 2	sc_t	6	3 msb are row index, 3 lsb are column index for t (intermediate) value in dpram
Milestone 2	cb_s	7	Column Block index for s value
Milestone 2	cb_sp	7	Column Block index for s' value
Milestone 2	rb_s	6	Row Block index for s value
Milestone 2	rb_sp	6	Row Block index for s' value

Milestone 2	fs_buf	16	A register to store a pair of s value after it has been read from sram
Milestone 2	LI_FS_addr_a0_cnt	7	Address counter for sram in fs lead in state
Milestone 2	addr_a1_T_cnt	7	Address counter for dpram in compute t lead in state
Milestone 2	sx_reg_0 , sx_reg_1(x in range 0-7)	8bits *16	2 sets of register to store s value. One set store one row of s, and another set store the next row of s
Milestone 2	tx_reg_0,tx_reg_1 (x in range 0-7)	32bits*16	2 sets of register to store t value. One set store one row of t, and another set store the next row of t
Milestone 2	ACC_reg_0	32	accumulation register
Milestone 2	FS_CSP_flag	1	A flag that will flip when one round of FS_CSP common case is finished
Milestone 2	WSP_CT_flag	1	A flag that will flip when one round of WSP_CT common case is finished
Milestone 2	finish_mega_flag	1	A flag that indicate when FSM should jump out from mega state loop
Milestone 2	select0	5bits*4	Multipliers Oprands will choose the correct value based on the value of select. (select0 is for multiplier1's operand, select1 is for multiplier2's operand chosen, and so on)
Milestone 2	select1		
Milestone 2	select2		
Milestone 2	select3		
Milestone 2	reg_0_multi	32	A register that store the output of multiplier 0
Milestone 2	reg_1_multi	32	A register that store the output of multiplier 1
Milestone 2	reg_2_multi	32	A register that store the output of multiplier 2
Milestone 2	reg_3_multi	32	A register that store the output of multiplier 3
Milestone 2	c_idx[3:0]	5bits*4	DCT matrix is passed into a mux. It will select correct value to output depend on c_idx
Milestone 2	address_a[1:0]	7	a register to store the address of port a (dpram0 & 1)
Milestone 2	address_b[1:0]	7	a register to store the address of port b (dpram0 & 1)
Milestone 2	write_data_a [1:0]	32	A register to store the data that will be input into the port a (dpram0 & 1)
Milestone 2	write_data_b [1:0]	32	A register to store the data that will be input into the port b (dpram0 & 1)

Milestone 2	write_enable_a [1:0]	1	write enable signal for port a(dpram0 & 1)
Milestone 2	write_enable_b [1:0]	1	write enable signal for port b(dpram0 & 1)

Testbench Verification

Testbench file was modified since the design save Y, U and V data at fixed offset in external SRAM for any sizes of the image. For milestone 1, the address offset for Y, U and V are 614,400, 768,000 and 844,800 respectively. For milestone 2, the address offset for post-DCT Y, U and V are 0, 307,200 and 460,800.

Steps for Run the Verification

Milestone 1 and Milestone 2 is done with Module-level Unit Testing.

16x16 Image – Milestone 1

```
# SRAM is now filled at    0.19us
#
# Starting Encoder at      500.17us
#
# Encoding finished at     525.11us
#
# Comparing TB and SW results
# Opening file "../data/fireball-0.dle" for Comparison
# ** Note: $stop      : ../tb/testbench.sv(156)
#   Time: 525170 ns  Iteration: 1  Instance: /TB
# Break in Module TB at ../tb/testbench.sv line 156
```

16x16 Image – Milestone 2

```
# Opening file "../data/fireball-0.dle" for initializing SRAM
# Finish initializing SRAM
#
# SRAM is now filled at    0.19us
#
# Starting Encoder at      500.17us
#
# Encoding finished at     525.61us
#
# Comparing TB and SW results
# Opening file "../data/fireball-0.d2e" for Comparison
# ** Note: $stop      : ../tb/testbench.sv(156)
#   Time: 525670 ns  Iteration: 1  Instance: /TB
# Break in Module TB at ../tb/testbench.sv line 156
```

640x480 Image – Milestone 1

```
# SRAM is now filled at      0.19us
#
# Starting Encoder at      500.17us
#
# Encoding finished at    22206.07us
#
# Comparing TB and SW results
# Opening file "../data/fractall-0.dle" for Comparison
# ** Note: $stop      : ../tb/testbench.sv(156)
#      Time: 22206130 ns  Iteration: 1  Instance: /TB
# Break in Module TB at ../tb/testbench.sv line 156
```

640x480 Image – Milestone 2

```
# *** Asserting the asynchronous reset ***
# *** Deasserting the asynchronous reset ***
#
# Opening file "../data/fractall-0.dle" for initializing SRAM
# Finish initializing SRAM
#
# SRAM is now filled at      0.19us
# |
# Starting Encoder at      500.17us
#
# Encoding finished at    27191.37us
#
# Comparing TB and SW results
# Opening file "../data/fractall-0.d2e" for Comparison
# ** Note: $stop      : ../tb/testbench.sv(156)
#      Time: 27191430 ns  Iteration: 1  Instance: /TB
# Break in Module TB at ../tb/testbench.sv line 156
```

Compilation Report (for Milestone 1 and 2):

Flow Status	Successful - Fri Mar 13 21:31:17 2026
Quartus Prime Version	19.1.0 Build 670 09/22/2019 SJ Lite Edition
Revision Name	project1
Top-level Entity Name	project1
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	4,447 / 114,480 (4 %)
Total registers	1996
Total pins	131 / 529 (25 %)
Total virtual pins	0
Total memory bits	8,192 / 3,981,312 (< 1 %)
Embedded Multiplier 9-bit elements	16 / 532 (3 %)
Total PLLs	1 / 4 (25 %)

Timing Analyzer:

	Fmax	Restricted Fmax	Clock Name	Note
1	54.77 MHz	54.77 MHz	clk_50	

Note: After Synthesized, quatus created some “false path”, which start from a register of milestone 1, and end in a register of milestone 2. It is impossible for a signal to travel in this path, as the FSM will not allow Milestone 1 and Milestone 2 run at the same time. They are different state. Thus, in order to get the real timing violation report, a SDC file is used to ignore any path that goes from milestone 1 to milestone 2, and vise versa.

Critical Path

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	1.496	top_state[0]	Milestone_1:Milestone_1_unit[U_buf[1]][3]	clk_50	clk_50	20.000	-0.072	18.430
2	1.609	top_state[2]	Milestone_1:Milestone_1_unit[U_buf[1]][3]	clk_50	clk_50	20.000	-0.072	18.317
3	1.636	top_state[1]	Milestone_1:Milestone_1_unit[U_buf[1]][3]	clk_50	clk_50	20.000	-0.073	18.289
4	1.656	Milestone_1:Milestone_1_unit[V_odd_shift_Reg[5]][3]	Milestone_1:Milestone_1_unit[V_buf[1]][4]	clk_50	clk_50	20.000	-0.088	18.254
5	1.658	top_state[0]	Milestone_1:Milestone_1_unit[U_buf[1]][5]	clk_50	clk_50	20.000	-0.072	18.268
6	1.659	top_state[0]	Milestone_1:Milestone_1_unit[U_buf[1]][2]	clk_50	clk_50	20.000	-0.072	18.267
7	1.659	top_state[0]	Milestone_1:Milestone_1_unit[U_buf[1]][6]	clk_50	clk_50	20.000	-0.072	18.267
8	1.660	top_state[0]	Milestone_1:Milestone_1_unit[U_buf[1]][1]	clk_50	clk_50	20.000	-0.072	18.266
9	1.663	top_state[0]	Milestone_1:Milestone_1_unit[V_buf[1]][4]	clk_50	clk_50	20.000	-0.086	18.249
10	1.664	top_state[0]	Milestone_1:Milestone_1_unit[U_buf[1]][4]	clk_50	clk_50	20.000	-0.072	18.262